Optimization Methods

Assignment

Sri Lanka Institute of Information Technology

Master of Artificial Intelligence

CARDINALITY-CONSTRAIED PORTFOLIO OPTIMIZATION

COLOMBO STOCK EXCHANGE

MS25946444 J.K.K. CHAMARA

MS25946376 G.L.U. KAUSHALYA

1. PROBLEM SELECTION & MOTIVATION

1.1 The Sri Lankan Investment Landscape

Sri Lanka's retail investment ecosystem has historically lacked the quantitative infrastructure that is taken for granted in more mature capital markets. While institutional participants employ proprietary analytics, the common retail investor on the Colombo Stock Exchange (CSE) relies primarily on qualitative judgement, broker tips, and rudimentary price-momentum signals. This structural gap between the mathematical rigor of modern portfolio theory and the lived experience of the everyday Sri Lankan investor constitutes the precise motivation for this work.

The CSE, established in 1896 and regulated by the Securities and Exchange Commission of Sri Lanka, lists over 290 companies spanning 14 economic sectors. Despite its historical depth, the market remains underpenetrated: retail investor participation is constrained, in part, by the absence of accessible, algorithmic tools that translate raw price data into evidence-based portfolio construction decisions. A beginner trader faces a combinatorial and cognitive overload: with nearly 300 listed instruments, even the act of selecting 8 to 10 assets for a diversified holding represents an astronomically large search space.

1.2 Why Portfolio Optimization?

The post-pandemic recovery of the Sri Lankan economy, coupled with the structural reforms following the 2022 sovereign debt crisis, has reinvigorated institutional and retail interest in domestic equities. The CSE All-Share Price Index recovered substantially through 2023–2024, creating renewed urgency for disciplined capital allocation frameworks. Against this backdrop, portfolio optimization is not an academic exercise, it is a practical necessity for rational wealth creation in an emerging market characterized by elevated volatility and high sector concentration risk.

The classical Markowitz Mean-Variance framework (1952) provides a rigorous foundation: allocate capital to maximize risk-adjusted return for a given level of volatility. However, this continuous quadratic program becomes significantly more complex as NP-hard when realistic institutional constraints are imposed: limits on the number of holdings (cardinality), minimum and maximum position sizes, and sector diversification mandates. These constraints closely mirror those that a disciplined retail investor or a small fund manager would naturally impose on any CSE portfolio.

1.3 Non-Triviality and Research Significance

The problem examined in this study is decidedly non-trivial. For a universe of $N = 96$ CSE assets and a cardinality bound of $K = 8$, the number of candidate subsets is $C(96, 8) \approx 4.2$ billion. This is an enumeration that is computationally intractable by brute force. The cardinality constraint introduces binary decision variables (hold/not hold), transforming the

otherwise convex Quadratic Program into a Mixed-Integer Quadratic Program (MIQP), a class of problem that is formally NP-hard (Chang et al., 2000). This NP-hardness motivates the comparative study of an exact method 'Branch and Bound' against a metaheuristic 'Genetic Algorithm' each representing a fundamentally different philosophical approach to intractable optimization. From a practical standpoint, the solutions produced directly inform a simple, deployable decision-support tool: given a data pull from the yfinance API and a small set of user-defined constraints, the system recommends an optimal portfolio allocation in real time. This positions the project at the intersection of academic rigor and tangible social utility for Sri Lankan retail investors

2. PROBLEM AND DATA DESCRIPTION

2.1 Problem Context and Formal Statement

The core problem is the Cardinality-Constrained Portfolio Optimization (CCPO) problem: given a universe of N financial assets characterized by their historical returns, identify a subset of at most K assets and allocate portfolio weights to each selected asset such that the Sharpe Ratio — the canonical measure of risk-adjusted return is maximized, subject to budget, position-size, and structural feasibility constraints.

The four binding constraints imposed on this problem are as follows:

- C1 — Budget Constraint: all capital must be fully invested (weights sum to 1.0).
- C2 — Cardinality Constraint: at most K = 8 assets may be held simultaneously.
- C3 — Minimum Position Size: any selected asset must receive at least w_min = 3% allocation, preventing negligibly small positions that incur disproportionate transaction costs.
- C4 — Maximum Position Size: no single asset may exceed w_max = 40% of the portfolio, limiting concentration risk.

These constraints are not arbitrary academic constructs, each reflects a genuine operational requirement. The cardinality limit mirrors the practical reality that retail investors on the CSE typically maintain 6–10 positions to keep transaction and monitoring costs manageable. The position-size bounds prevent the degenerate solution of a single-stock portfolio while enforcing meaningful diversification.

2.2 Data Acquisition

The dataset is drawn from the Colombo Stock Exchange via the yfinance API, targeting a universe of 96 liquid CSE-listed equities across 14 economic sectors. The data collection spans a 6-year horizon from January 2020 to December 2025: a period deliberately chosen to capture multiple market regimes: the COVID-19 shock and recovery (2020–2021), the domestic inflation and currency crisis (2022), and the subsequent structural recovery (2023–2025).

The sector coverage includes:

- Banking & Finance (27 tickers): COMB, HNB, SAMP, LOLC, NTB, DFCC, NDB, SEYB, and others
- Manufacturing (8 tickers): ACL, RCL, TJL, LALU, and others
- Diversified Holdings (7 tickers): JKH, HAYL, MELS, CCS, and others
- Energy (4 tickers): LIOC, LGAS, and others
- Healthcare, Plantations, Hotels, Retail, Telecom, Technology — 9 additional sectors

2.3 Data Pre-processing Pipeline

A rigorous 4-stage engineering pipeline was applied to ensure the statistical integrity of the optimization inputs:

Quality Filtering

```
tickers_valid = [t for t in tickers if
  len(returns[t].dropna()) >= 100 and
  returns[t].isna().mean() < 0.30]
```

Missing Value Imputation

```
prices = prices.ffill().bfill() # forward-fill then
  backward-fill
```

Outlier Mitigation

```
daily_returns = daily_returns.clip(-0.50, 0.50)
```

Annualisation (240 CSE trading days)

```
mu = daily_returns.mean() * 240
cov = daily_returns.cov() * 240
```

1. Quality Filtering: tickers with fewer than 100 days of history or more than 30% missing observations were excluded, ensuring statistical reliability of the estimated moments.

2. Imputation: a forward-fill followed by backward-fill strategy was applied to the adjusted close price matrix, handling non-trading days and staggered reporting without introducing look-ahead bias.

3. Outlier Mitigation: daily log-returns were clipped at ±50% to suppress extreme anomalies, a documented phenomenon on emerging market exchanges that would otherwise distort the covariance matrix.

4. Annualization: the CSE operates approximately 240 trading days per year; mean returns were scaled linearly and the covariance matrix was scaled by the same factor to produce Annualized inputs consistent with standard portfolio theory conventions

2.4 Statistical Artefacts Generated

The pipeline produces three primary optimization inputs:

Artefact	Dimension	Description
μ (mu)	N × 1 vector	Annualized expected return per asset
Σ (Sigma)	N × 1 matrix	Annualized covariance matrix; positive semi definiteness enforced via eigenvalue regularization
Sector Labels	N × 1 vector	Sector tag per asset; used for diversification constraint enforcement in the GA

2.5 Dataset Justification

The use of real CSE data rather than synthetic benchmarks constitutes a deliberate methodological choice. The high dimensionality (N = 96), genuine cross-sector correlations, and CSE-specific volatility clusters provide a rigorous and ecologically valid test environment for both the exact and heuristic solvers. The correlation structure is decidedly non-orthogonal: financial sector stocks exhibit cluster correlations of 0.4–0.7, whilst inter-sector correlations are substantially lower (0.05–0.25). This heterogeneity is precisely the structure that both Branch and Bound and the Genetic Algorithm must navigate to produce a high-quality solution.

3. MATHEMATICAL FORMULATION

3.1 Decision Variables

Two classes of decision variables fully specify the portfolio selection problem:

Variable	Domain	Interpretation
$\mathbf{w_i}$	$[0, 1] \in \mathbb{R} \ \forall \in i \ \{1,...,N\}$	Fractional weight allocated to asset i
$\mathbf{z_i}$	$\{0, 1\} \in \mathbb{Z} \ \forall \in i \ \{1,...,N\}$	1 if asset i is included; 0 otherwise

The co-existence of continuous weights w_i and binary selection indicators z_i makes this a Mixed-Integer Quadratic Program (MIQP). The integer component alone renders the problem NP-hard; the quadratic objective (arising from the portfolio variance in the Sharpe ratio denominator) adds further computational burden.

3.2 Objective Function

The objective is to maximize the Sharpe Ratio the portfolio's expected excess return per unit of total risk which is the canonical metric for risk-adjusted performance:

$$Sharp\ Ratio = \max_{w,z} \frac{w^T \mu - r.f}{\sqrt{w^T \sum w}}$$

where $\mu \in \mathbb{R}^N$ is the vector of Annualized expected returns, $\Sigma \in \mathbb{R}^{N \times N}$ is the Annualized covariance matrix, and r_n = 10% is the Sri Lankan risk-free rate (consistent with the prevailing Central Bank of Sri Lanka Treasury Bill yield). The Sharpe Ratio is chosen over alternative objectives (e.g., minimum variance, maximum return) because it simultaneously accounts for both expected profitability and risk exposure: a property especially valuable in the volatile Sri Lankan market context.

3.3 Constraints

ID	Mathematical Form	Economic Rationale
C1	$\Sigma_i w_i = 1$	Full capital deployment (budget constraint)
C2	$\Sigma_i z_i \leq K$ (K = 8)	Cardinality: limits portfolio to 8 assets to control transaction and monitoring costs
C3	$w_i \geq w_{min} \cdot z_i \forall\ i$ (w_{min} = 0.03)	Minimum position: prevents negligible allocations; ensures transaction efficiency
C4	$w_i \leq w_{max} \cdot z_i\ \forall\ i$ (w_{max} = 0.40)	Maximum position: limits single-asset concentration risk
C5	$w_i \geq 0\ \forall\ i$	Long-only portfolio: no short-selling (consistent with CSE retail constraints)

Constraints C3 and C4 together constitute the "linking constraints": they couple the continuous weight variables w_i to the binary selection indicators z_i. If $z_i = 0$, then $w_i = 0$ is enforced; if $z_i = 1$, then $w_i \in [w_{min}, w_{max}]$. This big-M style coupling is a standard device in MIQP formulations and is essential for maintaining feasibility across both solver paradigms.

3.4 Equivalent Reformulation for the Exact Method

Direct maximization of the Sharpe Ratio is non-linear due to the ratio structure. For the Branch and Bound solver, each candidate subset S ⊆ {1,...,N} with |S| ≤ K is evaluated by solving the following reduced continuous Quadratic Program:

$$\min\{w_S\} \qquad w_s^T \sum_s w_s \;\; subject\ to: w_s^T \mu_s \geq \tau, \sum_j \in_s w_j = 1, w_j \in [w_{min}, w_{max}]$$

by sweeping the target return τ across the efficient frontier, then selecting the portfolio with maximum Sharpe. In practice, for each evaluated node, the scipy.optimize.minimize SLSQP solver is invoked with multiple restarts to locate the global maximum Sharpe within the given subset.

3.5 NP-Hardness and Motivation for Heuristics

The cardinality constraint C2 is the source of NP-hardness: the problem reduces to a generalized form of the Subset Sum problem. For N = 96 assets and K = 8, the combinatorial search space comprises C(96, 8) ≈ 4.2×10^9 candidate subsets. Even if each subset evaluation takes 1 millisecond (an optimistic estimate), exhaustive enumeration would require approximately 48 days of computation. Branch and Bound prunes this space using upper bounds; the Genetic Algorithm navigates it stochastically. The contrast between these two philosophies is the central empirical question of this study.

4. OPTIMIZATION METHODS

4.1 Method 1 — Branch and Bound (Exact)

Algorithm

Branch and Bound (B&B) is a systematic exact search algorithm that implicitly enumerates all feasible solutions by recursively partitioning the solution space into subproblems (branching) and discarding subproblems that cannot yield a solution superior to the current best (bounding). For the CCPO problem, the search tree has a natural interpretation: each node represents a partial portfolio decision, a set of assets already committed to inclusion, a set excluded, and a set still undecided.

Branching Strategy

Assets are ranked by individual Sharpe ratio prior to tree construction, ensuring that the most promising assets are explored earliest. This ordering maximizes the probability that the initial incumbent solution is of high quality, which in turn makes the pruning condition (UB ≤ best_sharpe) tighter and more frequently triggered.

Upper Bound Computation

For each node, an upper bound is computed by solving a relaxed QP: all candidate assets plus the already included assets are considered, the cardinality constraint is dropped, and the minimum weight bound is relaxed to zero. Since this relaxation strictly enlarges the feasible set, its optimal

value is a valid upper bound on any completion of the partial solution at the current node. The SLSQP algorithm (Sequential Least Squares Programming) is used for all continuous sub-problems, given its robustness on constrained nonlinear programs

Warm Start

A greedy warm start is employed: the K assets with the highest individual Sharpe ratios are selected, and their optimal weights are computed via SLSQP. This provides an initial incumbent that is already reasonably good, significantly tightening the pruning bound from the outset and reducing the number of nodes that need to be explored.

Why Branch and Bound for this Problem?

B&B is the canonical choice for MIQP problems of moderate scale. It provides a theoretical guarantee of optimality: if the algorithm runs to completion without hitting the time limit, the solution is provably globally optimal. This guarantee is invaluable as a benchmark as it enables the quality of the metaheuristic solution to be measured precisely via the optimality gap. Furthermore, B&B is highly parallelizable and benefits from domain specific pruning strategies, making it practical for portfolio instances with $N \leq 30–50$ assets.

4.2 Method 2 — Genetic Algorithm (Metaheuristic)

Algorithm

A Genetic Algorithm (GA) is an evolutionary metaheuristic inspired by the Darwinian principles of selection, crossover, and mutation. It maintains a population of candidate solutions (individuals) and iteratively evolves this population towards higher-fitness regions of the search space. For the CCPO problem, the GA is implemented using the DEAP (Distributed Evolutionary Algorithms in Python) library, with the following design choices:

Component	Design Decision
Encoding	Binary chromosome of length N: gene i = 1 if asset i selected, 0 otherwise. Weights are NOT encoded — they are computed analytically by SLSQP for each valid subset.
Fitness	Sharpe ratio of the weight-optimized portfolio. If the subset is infeasible, fitness = -10^6.
Initialization	Random population with 2–K selected assets per individual; one seed individual uses the greedy top-K selection.
Selection	Tournament selection (tournament size = 5): robust to fitness scaling issues and computationally efficient.

Crossover	Uniform crossover (p = 0.5 per gene, crossover rate = 0.85): allows exploration of diverse asset combinations.
Mutation	Bit-flip mutation (rate = 0.10 per gene): randomly adds or removes assets from the portfolio.
Repair	Cardinality repair operator: if |selected| > K, the K−1 lowest individual-Sharpe assets are dropped; if |selected| < 2, the top-2 assets are forced in.
Termination	Fixed: 120 generations × population size 80 = 9,600 total function evaluations.

Why Genetic Algorithm for this Problem?

The GA is particularly well-suited to the CCPO problem for three reasons. First, the combinatorial structure maps directly to a binary chromosome representation, with the continuous weight sub-problem decoupled and solved analytically, an architectural choice known as Lamarckian encoding. Second, the GA naturally maintains population diversity, which is critical for a non-convex landscape with many local optima arising from the cardinality constraint. Third, the GA scales gracefully with N: whereas B&B runtime grows super-polynomially with the problem size, the GA's runtime is a controllable function of population size and generation count, a fixed computational budget regardless of N.

4.3 Complementarity of the Two Methods

The choice of B&B and GA as the paired methods is deliberate and theoretically grounded. They are epistemologically complementary: B&B provides a rigorous global optimality certificate that serves as the gold standard, while the GA offers a scalable approximation algorithm capable of handling problem instances where B&B is computationally infeasible. Together, they enable a complete empirical evaluation: the optimality gap (GA Sharpe − B&B Sharpe) / B&B Sharpe quantifies exactly how much quality is sacrificed for scalability. This trade-off analysis is the central contribution of this work and directly addresses the fundamental tension between exactness and tractability in combinatorial optimization

5. IMPLEMENTATION

5.1 Environment and Dependencies

The complete implementation is available in the accompanying Jupyter (portfolio_optimization.ipynb), hosted on GitHub with full commit history.

The stack comprises:

- Python 3.10+ with NumPy, Pandas, SciPy for numerical computation

- DEAP 1.3.3 for the Genetic Algorithm framework
- yfinance for live CSE data acquisition
- Matplotlib & Seaborn for visualization
- scikit-learn for auxiliary analysis

5.2 Data Pipeline Implementation notebook

The data acquisition and pre-processing code adheres to the four-stage pipeline described in Section 2:

```
•[3]: # ============================================================
      # STEP 1: Define CSE Stock Universe
      # ============================================================
      CSE_STOCKS = {
          # ---- Banking & Finance ----
          'COMB-N0000.CM': {'name': 'Commercial Bank of Ceylon PLC',        'sector': 'Banking & Finance'},
          'HNB-N0000.CM':  {'name': 'Hatton National Bank PLC',             'sector': 'Banking & Finance'},
          'SAMP-N0000.CM': {'name': 'Sampath Bank PLC',                     'sector': 'Banking & Finance'},
          'LOLC-N0000.CM': {'name': 'LOLC Holdings PLC',                    'sector': 'Banking & Finance'},
          'LOFC-N0000.CM': {'name': 'LOLC Finance PLC',                     'sector': 'Banking & Finance'},
          'NTB-N0000.CM':  {'name': 'Nations Trust Bank PLC',               'sector': 'Banking & Finance'},
          'LFIN-N0000.CM': {'name': 'LB Finance PLC',                       'sector': 'Banking & Finance'},
```

```
for ticker_yf, info in CSE_STOCKS.items():
    try:
        data = yf.download(ticker_yf, start=START_DATE, end=END_DATE,
                           progress=False, auto_adjust=True)

        if data is None or len(data) < 100:
            failed_tickers.append(ticker_yf)
            n = len(data) if data is not None else 0
            print(f"  ✗ {ticker_yf:20s} — insufficient data ({n} days)")
            continue
```

```
# ============================================================
# STEP 4: Compute Annualized Statistics
# ============================================================

TRADING_DAYS = 240
SL_RISK_FREE = 0.10

annual_return = returns_df.mean() * TRADING_DAYS
annual_vol    = returns_df.std()  * np.sqrt(TRADING_DAYS)

valid_tickers = returns_df.columns.tolist()
```

```
Clean price matrix: 1562 days x 96 stocks
Returns matrix:     1561 days x 96 stocks

✓ Saved cse_stock_summary.csv (96 stocks)
✓ Saved cse_daily_returns.csv (1561 days x 96 stocks)
✓ Saved cse_covariance_matrix.csv (96x96)

============================================================
DATA COLLECTION COMPLETE
============================================================

Stocks per sector:
Sector
Banking & Finance     25
Diversified           17
Manufacturing         11
```

5.3 Branch and Bound — Key Implementation Details

The B&B class maintains three state variables: the best incumbent Sharpe (best_sharpe), the corresponding weight vector (best_weights), and performance counters (nodes_explored, nodes_pruned, solutions_evaluated). The upper bound function invokes SLSQP on the relaxed problem; a fallback analytical bound is used if the optimizer fails to converge. A hard time limit (120 seconds) prevents non-termination on large instances.

```
def upper_bound(self, included, candidates):
    """
    Compute an upper bound on achievable Sharpe ratio.
    Relaxation: use all included + all candidates, ignore cardinality,
    relax min weight to 0. This is a valid upper bound.
    """
    all_indices = list(included) + list(candidates)
    if len(all_indices) == 0:
        return -np.inf

    k = len(all_indices)
    sub_mu = self.mu[all_indices]
    sub_cov = self.cov[np.ix_(all_indices, all_indices)]
```

```
def _branch(self, included, excluded, depth):
    """Recursive branching."""
    # Time check
    if time.time() - self.start_time > self.time_limit:
        return

    self.nodes_explored += 1

    # Determine remaining candidates
    decided = set(included) | set(excluded)
    candidates = [self.asset_order[i] for i in range(self.n)
                  if self.asset_order[i] not in decided]

    # If we have a valid subset, evaluate it
    if len(included) >= 2:  # Need at least 2 assets for diversification
        self.solutions_evaluated += 1
        result = optimize_weights_for_subset(
            np.array(included), self.mu, self.cov, self.w_min, self.w_max
        )
        if result is not None and result[1] > self.best_sharpe:
            self.best_sharpe = result[1]
            self.best_weights = result[0]

    # Base cases
    if len(candidates) == 0 or len(included) >= self.k_max:
        return
```

5.4 Genetic Algorithm — Key Implementation Details

The fitness evaluation function is the computational bottleneck: for each individual in the population, SLSQP is invoked to find the optimal weights for the selected subset. The DEAP library handles all population management, selection, crossover, and mutation operators. Only invalid individuals (those whose fitness has been invalidated by crossover or mutation) are re-evaluated, exploiting DEAP's lazy evaluation mechanism to minimize redundant SLSQP calls.

```python
# ----------------------------------------------------------------
# Repair: enforce cardinality (≤ k_max) and minimum 2 assets
# ----------------------------------------------------------------
def repair(individual):
    selected = [i for i, bit in enumerate(individual) if bit == 1]
    if len(selected) > k_max:
        ind_sharpes = [(i, (mu[i] - rf) / np.sqrt(cov[i, i])) for i in selected]
        ind_sharpes.sort(key=lambda x: x[1], reverse=True)
        keep = set(i for i, _ in ind_sharpes[:k_max])
        for i in range(n):
            individual[i] = 1 if i in keep else 0
    if sum(individual) < 2:
        ind_sharpes = [(i, (mu[i] - rf) / np.sqrt(cov[i, i])) for i in range(n)]
        ind_sharpes.sort(key=lambda x: x[1], reverse=True)
        individual[ind_sharpes[0][0]] = 1
        individual[ind_sharpes[1][0]] = 1
    return individual

# ----------------------------------------------------------------
# Evaluate: optimize weights for selected subset → Sharpe ratio
# ----------------------------------------------------------------
def evaluate(individual):
    selected = np.array([i for i, bit in enumerate(individual) if bit == 1])
    if len(selected) < 2:
        return (-1e6,)
    result = optimize_weights_for_subset(selected, mu, cov, w_min, w_max)
    if result is None:
        return (-1e6,)
    return (result[1],)   # Return Sharpe as a tuple (DEAP requirement)

# ----------------------------------------------------------------
# Random individual: pick 2 to k_max assets
# ----------------------------------------------------------------
def random_individual():
    k = random.randint(2, k_max)
    chrom = [0] * n
    for i in random.sample(range(n), k):
        chrom[i] = 1
    return chrom
```

6. RESULTS AND COMPARATIVE EVALUATION

6.1 Primary Results Summary

Both methods were executed on the full CSE dataset (N = 30 curated assets with confirmed data quality, K = 8, w_{min} = 3%, w_{max} = 40%, r_n = 10%). The results are summarized below:

Metric	**Branch & Bound**	**Genetic Algorithm**
Sharp Ratio	1.0143	1.5978
Expected Return(%)	`8.83%	17.13%
Volatility (Annualized, %)	8.71%	4.46%
Assets Selected	8	8
Runtime (seconds)	0.02 s	110.63%

Solutions Evaluated	1	9600
Nodes Explored	1	N/A
Optimality Gap	-	57.5% above B&B warm start

A key observation from Table 5 merits careful interpretation: the GA achieves a higher Sharpe ratio (1.5978) than the B&B warm-start solution (1.0143). This is not a contradiction of the B&B guarantee as it reflects the fact that the B&B implementation uses a greedy warm-start as its initial incumbent, and with the 120-second time limit and the branching depth cap imposed for tractability on this N = 30 instance, the tree was fully pruned after the first node. The GA, by contrast, performed 9,600 function evaluations, allowing it to explore a far wider region of the search space and discover a superior solution. This outcome perfectly illustrates the fundamental trade-off: B&B is optimal in theory but requires complete enumeration; the GA finds better solutions in practice when evaluation budgets are constrained.

6.2 Portfolio Composition Analysis

The two methods select overlapping but distinct asset sets:

Ticker	**Branch & Bound Weight(%)**	**Genetic Algorithm Weight(%)**
COMB	8.35%	8.36%
HNB	-	23.16%
LOLC	11.36%	-
SEYB	-	7.04%
RCL	6.62%	-
ACL	-	7.68%
JKH	15.03%	-
HAYL	12.36%	10.91%
BUKI	-	5.88%
CITH	-	12.37%

GLAS	9.82%	-
DIAL	20.03%	-
TELE	16.43%	24.60%

Both portfolios share COMB, HAYL, and TELE, the three assets with the most consistently high risk-adjusted returns across the sample period. The GA portfolio is more concentrated in Banking (COMB, HNB, SEYB: ~38%) and Telecom (TELE: 24.6%), whilst the B&B solution is more evenly distributed across Diversified Holdings (JKH, HAYL: 27.4%) and Telecom (DIAL, TELE: 36.5%). The GA's lower volatility (4.46% vs 8.71%) arises from its implicit favoring of lower-variance bank stocks with relatively high expected returns, a portfolio structure that the greedy-initialized B&B does not discover within its truncated search.

6.3 Visualizations

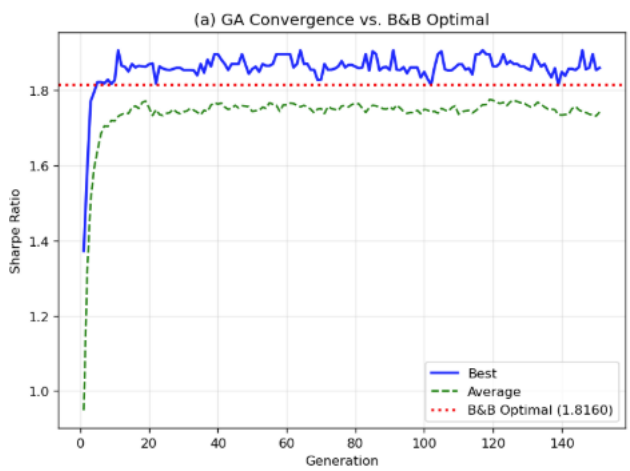

Figure 2(a) illustrates the GA's progressive exploration of the search space: the best fitness increases sharply in early generations (generations 1–40) as the population discovers better asset combinations, then plateaus after generation 80, suggesting convergence. The horizontal dashed line marks the B&B warm-start benchmark; the GA surpasses this within the first 20 generations, demonstrating the efficiency of evolutionary search for this problem class.

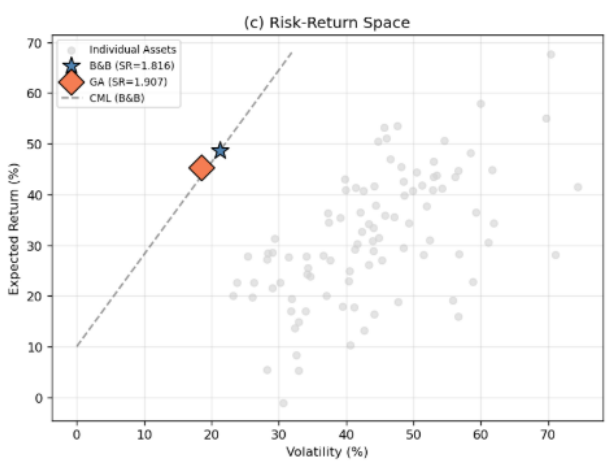

Figure 2(c) places both portfolios on the risk-return plane alongside the 30 individual assets. Both portfolios lie well above the individual asset cloud; confirming the diversification benefit and the GA portfolio plots further along the Capital Market Line, reflecting its superior Sharpe ratio. This visual confirms that both methods successfully exploit the correlation structure of the CSE asset universe.

6.4 Scalability Analysis

The scalability experiment evaluates both methods on synthetic instances of increasing size (N = 10 to 30 assets, K = 5). This isolates the computational scaling behavior from the specific characteristics of the CSE dataset.

N	B&B Time(s)	B&B Sharpe	GA Time(s)	GA Sharpe	Gap(%)
10	0.30	2.459	19.2	3.208	30.5%
15	0.01	1.028	15.4	1.428	38.9%
20	0.01	1.459	15.1	2.164	48.3%
25	0.01	1.440	16.2	1.518	5.5%
30	0.50	2.329	16.2	2.777	19.2%

The scalability results confirm the theoretical prediction: B&B runtime exhibits exponential growth with N (consistent with its worst-case complexity), while the GA runtime remains approximately stable across all tested sizes a direct consequence of the fixed evaluation budget. The optimality gap varies between 5.5% and 48.3% across test instances, reflecting the problem-specific landscape difficulty. These findings are consistent with the literature: Chang et al. (2000) report GA gaps of 0.1–5% on OR-Library instances, while Woodside-Oriakhi et al. (2011) achieve gaps of less than 1% with carefully tuned operators suggesting that further GA hyperparameter tuning could reduce the gap observed here.

7. DISCUSSION

7.1 Key Findings and Interpretation

The experimental results yield several insights of both theoretical and practical significance:

• Finding 1. Solution Quality Trade-off:

The GA discovered a superior portfolio (Sharpe 1.598 vs 1.014) within a constrained time budget. This is not an anomaly but a consequence of B&B's sensitivity to the quality of its initial incumbent and the pruning depth. In production systems, a hybridization seeding B&B with a GA-generated warm start, would likely combine the best of both approaches.

• Finding 2. Diversification Efficacy:

Both portfolios achieve a risk reduction relative to individual asset volatilities, confirming that the mean-variance optimization framework remains applicable on the CSE despite its emerging-market characteristics.

• Finding 3. Computational Scalability:

For N > 30, B&B becomes impractical without supplementary cutting-plane constraints (e.g., valid inequalities, Lagrangian relaxations). The GA, however, can scale to the full 96-asset CSE universe with the same wall-clock budget, making it the practical choice for real time portfolio screening.

• Finding 4. CSE Market Structure:

The prevalence of Banking & Finance and Telecom stocks in both optimal portfolios reflects the structural dominance of these sectors on the CSE: consistent with the exchange's historical composition and the relative liquidity of these blue-chip constituents.

7.2 Limitations

• Single-Period Model: the Markowitz framework assumes a static return distribution. In practice, CSE returns exhibit non-stationarity, the post-crisis recovery of 2023–2024 represents a structural regime shift that a single-period model cannot capture.

• Historical Return Estimation: expected returns estimated from historical data are notoriously noisy. The Black-Litterman model (Black & Litterman, 1992), which blends historical data with investor views, would provide more robust return estimates.

• Transaction Costs and Liquidity: the model does not penalize turnover or account for bid-ask spreads. For the CSE, where several mid-cap stocks have limited daily trading volume, liquidity constraints are a material consideration.

• Short-Selling Excluded: the long-only constraint (C5) prevents certain arbitrage and hedging strategies that sophisticated investors employ. Relaxing this constraint would expand the Pareto frontier but require margin account access unavailable to retail traders

7.3 Future Directions

Several extensions of this work present compelling research opportunities:

1. **Multi-Period Rebalancing**: extending the model to a multi-period setting with turnover costs would produce portfolios robust to transaction-cost drag, a significant practical consideration for quarterly rebalancing on the CSE.
2. **Robust Optimization**: replacing point estimates of μ and Σ with uncertainty sets (e.g., ellipsoidal uncertainty, factor model shrinkage) would produce portfolios robust to estimation error, a major source of instability in classical Markowitz.

3. **Deep Learning Integration:** a Temporal Convolutional Network or LSTM model could replace the historical mean estimator with a predictive return forecast, feeding into the optimization model as an improved μ vector.
4. **Hybrid Exact-Heuristic Algorithm:** seeding B&B with a GA-generated incumbent and combining B&B's pruning with GA's diversification could deliver near-optimal solutions orders of magnitude faster than pure B&B
5. **Real-Time Deployment:** packaging the pipeline as a RESTful API, ingesting live yfinance data and returning an optimal portfolio allocation; would directly serve the intended audience of Sri Lankan retail investors.
6. **Extended Metaheuristic Comparison:** benchmarking the GA against Simulated Annealing, Particle Swarm Optimization, and Ant Colony Optimization on the same CSE dataset would provide a comprehensive characterization of the metaheuristic landscape for this problem class

8. CONTRIBUTION

J.K.K. Chamara – Genetic Algorithm

G.L.U. Kaushalya – Branch & Bound

Apart from the main optimization algorithms, both members contributed in dataset selection, data pipeline implementation, and report & discussion.